

Allowing contract predicates on non-first declarations

Abstract

This paper proposes allowing contract predicates on non-first declarations for a member function. The rationale of doing so is, in a nutshell, to allow preconditions and postconditions to be implementation details. This thus also introduces the ability to redeclare a member function with a non-defining redeclaration.

Rationale

How is this a bug fix?

There is a desire to write programs where contract checks are implementation details. When they aren't, work-arounds will be written. We'd prefer having direct support for contract checks as implementation details, rather than necessitating the work-arounds.

Why this feature is deemed necessary

The support for contract checking adopted for C++20 allows decorating function declarations with annotations for the function's pre- and postconditions for both free functions and member functions of a class. The current WP requires the contract annotations for either case to be attached to the first function declaration.

There are users that do not want to expose a contract checking mechanism in their class interfaces; there may be an English contract, but the language-level contract enforcement mechanism is an implementation detail that function declarations in a class definition should not expose.

Annotating the declaration in a typical scenario of having the declaration in a header file and the definition in a source file allows the contract checking annotations to be visible in more than one translation unit to compilers, static analyzers, and other tools working on per-TU basis. For a human reader, however, in case a proper English contract is provided in form of a comment, the contract-checking annotations only serve to expose the implementation details of which contract checks are performed and which level they are performed at. This not only distracts from the properly defined full contract, but also increases the overall length of a function declaration, especially if the lists of contract-checking annotations are long and many functions are declared within the same (class or namespace) scope. In addition, it is currently not possible to provide a fully insulated implementation of a function with regard to pre- and postcondition checks – any change in the contract annotations would necessarily require recompilation of all clients of a function or class.

It is probable that some would suggest that such use cases could be handled with contract assertions in function bodies. The reason why this paper proposes allowing contract predicates on non-first declarations is that

- In some cases, there is no body to put an assertion in; defaulted functions and pure virtual functions are examples of this.
- For postconditions, postconditions are guaranteed to be checked (when checking is enabled in the first place) at function exit; relying on contract assertions is more error-prone.

Furthermore, such an approach either forces to provide the implementation inline or loses the advantages of having the contract-checking annotations in the header file.

Usage examples

We basically propose being able to do this (case #1):

```
struct X
{
    void f();
};

void X::f() [[expects: foo]]
{
    ...
}
```

In addition, though, what is also proposed is this (case #2):

```
struct Y
{
    void f();
};
void Y::f() [[expects: foo]];

// definition possibly in a separate translation unit
void Y::f() [[expects: foo]]
{
    ...
}
```

Implementation impact, semantic restrictions

Allowing contract predicates on a non-first declaration probably means that in some such cases, the checking code can't be laid down at the call site unless the definition is also seen by the compiler.

This leads to an implementation concern about being able to decide whether all contract-checking is always done at the definition site, or whether it's possible to allow the caller to do that.

Based on an explanation by an implementation vendor, what we need to do is as follows:

1. If a class definition has a member function declaration *without* a contract on it, contract checking can be done at the call site if and only if the caller sees another declaration with a contract on it and the definition also sees such a declaration (possibly being one itself).
2. If a class definition has a member function declaration *with* a contract on it, the contract checking can be done at the call site regardless of other declarations.
3. If the member function declaration in the class definition did not have a contract, and an outside-of-class declaration visible to the caller has a contract, and there is no declaration visible at the site of the definition with a contract on it, the program is ill-formed, no diagnostic required.
4. If a function declaration *doesn't* have a contract on it, contract checking can be done at the call site if and only if the caller sees another declaration with a contract on it and the definition also sees such a declaration (possibly being one itself).
5. If there exists a function declaration with a contract on it visible to the caller, and there is no declaration with a contract visible at the site of the definition, the program is ill-formed, no diagnostic required.

Out of class member function declarations

In order to support case #2, member functions declarations should be allowed outside of the class definition.

```
struct A
{
    void f();
};
void A::f() [[expects: foo]];
```

The proposal does not suggest allowing redeclarations within the class definition.

```
struct B
{
    void f();
    void f() [[expects: foo]]; //error, redeclaration within class definition
}
```

Suggested rules for default arguments on out of class member function declarations are the same as rules for default arguments on non-member function declarations.

```
struct C
{
    void foo(int i, int j, int k = 3);
};

void C::foo(int i = 1, int j, int k); // error: no previous default argument for parameter j
void C::foo(int i, int j = 3, int k); // ok
void C::foo(int i, int j, int k = 3); // error: cannot redefine, even to same value
```

Suggested inline specifier rules for out of class member function declarations are the same as rules for the inline specifier when applied to non-member functions.

```
struct D1
{
    void foo();
};

inline void D1::foo();
void D1::foo(){} // ok, D1::foo is inline

struct D2
{
    void foo();
};

void D2::foo(){}
inline void D2::foo(); // error, first declaration as inline after definition
```

Implementation experience

This proposal, along with many other things, has been implemented in <https://gitlab.com/lock3/gcc-new>. Notably, that implementation doesn't place the semantic requirement on non-member functions.

Wording

In [dcl.fct.default]/6, modify as follows:

Except for member functions of class templates, the default arguments in a member function ~~definition~~**declaration** that appears outside of the class definition are added to the set of default arguments provided by the member function declaration in the class definition;

In [class.mfct]/1, remove the redeclaration restriction:

A member function ~~definition~~**declaration** that appears outside of the class definition shall appear in a namespace scope enclosing the class definition. A member function **declaration** shall appear in a class definition only once. ~~Except for member function definitions that appear outside of a class definition, and except for explicit specializations of member functions of class templates and member function templates (12.8) appearing outside of the class definition, a member function shall not be redeclared.~~

In [class.mfct]/2, modify as follows:

An inline member function (whether static or non-static) may also be defined outside of its class definition provided ~~either its declaration in the class definition or its definition outside of the class definition~~ **there exists a declaration that declares the function as inline or constexpr.**

In [class.mfct]/4, modify as follows:

If the ~~definition~~**declaration** of a member function is lexically outside its class definition, the member function name shall be qualified by its class name using the :: operator. [Note: A name used in a member function ~~definition~~**declaration** (that is, in the *parameter-declaration-clause* including the default arguments (9.2.3.6) or in the member function body) is looked up as described in 6.4. — end note]

In [dcl.attr.contract.cond]/1, modify as follows:

A contract condition is a precondition or a postcondition. ~~If~~ **the** first declaration of a function **has a contract condition, it shall specify all contract conditions (if any) of the function.** For a non-member function, all declarations that specify a contract condition shall specify the same list of contract conditions. For a member function, a member function redeclaration outside a class definition may specify contract conditions different from the ones in the declaration inside a class definition if the declaration inside a class definition had no contract conditions. Subsequent declarations shall either specify no contract conditions or the same list of contract conditions; no diagnostic is required if corresponding conditions will always evaluate to the same value. ~~For a non-member function,~~ **the** list of contract conditions of a function shall be the same if the declarations of that function **that contain contract conditions** appear in different translation units; no diagnostic required. For a member function, the list of contract conditions of a function shall be the same if the redeclarations outside a class definition of that function appear in different translation units; no diagnostic required. [Note: a declaration in a class definition and a declaration outside the class definition may have different lists of contract conditions. --end note] If a definition of a function is reachable at the point of its first declaration that contains contract conditions, the program is ill-formed; no diagnostic required.